

Industry Project
Title: Online Event Booking and
Management System

Name of the Company: TCS-ION

Name of the Institute: AKS UNIVERSITY, SATNA

B. Tech (CSE) 8th Semester

Submitted by

AYUSH CHAURASIYA (B2255R10106007)

TABLE OF CONTENT

- 1 Acknowledgements
2. Objective and Scope
3. Problem Statement
4. Existing Approaches
5. Approach / Methodology -
6. Workflow
7. Assumptions
8. Implementation
9. Solution Design
10. Challenges & Opportunities
11. Reflections on the project
12. Recommendations
13. Outcome / Conclusion
14. Enhancement Scope
15. Link to code and executable file
16. Research questions and responses
17. References

Acknowledgements

The development of the Online Event Booking and Management System involved building a complete working application where each part—frontend screens, backend APIs, and database—was implemented and connected step by step. This project was not just about writing code, but about making sure every user action, such as logging in or booking a ticket, correctly flows through the system and updates the database in a controlled way.

During the implementation, special attention was given to how different roles interact with the system. The separation between Guest, User, Organizer, and Admin was carefully handled so that each role only sees and performs actions relevant to them. For example, while a Guest can only view events, an Organizer has access to a separate dashboard (`organizer-dashboard.html`) where new events can be created, and an Admin uses `admin.html` to approve those events before they become visible.

Another important aspect of the project was maintaining consistency between frontend and backend. The frontend, written in HTML, CSS, and JavaScript, sends requests using the fetch API, while the backend processes those requests using Express routes like `auth.js`, `events.js`, `bookings.js`, and `admin.js`. These modules were designed in a way that each handles a specific part of the system, making the application easier to manage and extend.

The use of JWT authentication ensured that once a user logs in, their identity is securely maintained across requests without needing to repeatedly log in. This allowed features like booking tickets and accessing dashboards to be protected and role-specific.

Overall, the project required careful coordination between multiple components, and completing it provided practical experience in handling real-world application flow, data validation, and role-based system design.

Objective and Scope

The main objective of this project was to build a functional and structured event booking system where users can discover events and book tickets, while organizers and administrators manage the lifecycle of those events through controlled workflows.

The system was designed with clear boundaries on what each role can do:

A Guest user can open the application and view a list of available events without logging in. This is handled by fetching only approved events from the backend.

A User can register through the registration page, log in using the login page, and then book tickets for events. These actions involve interaction with the authentication routes and booking APIs.

An Organizer has access to a dedicated interface (organizer-dashboard.html) where they can create new events by submitting details such as title, description, date, location, and available seats. These events are not immediately visible; instead, they are stored with a Pending status.

An Admin accesses admin.html, where all pending events are listed. The admin can approve or reject events, and only approved events become visible to normal users.

The scope of the project is limited to:

Event creation and approval workflow

Secure user authentication using JWT

Ticket booking with seat validation

Role-based dashboards

The system does not include external integrations like payment gateways or email notifications, keeping the focus on core booking logic and system design.

Problem Statement

In many real-world scenarios, event booking is handled manually or through loosely organized systems. This creates several issues that this project aims to solve through a structured application.

One of the main problems is the lack of a centralized system where events can be created, reviewed, and published. Without this, organizers may share event details through messages or calls, and users may not have a reliable way to discover all available events in one place.

Another issue is lack of validation during booking. In manual systems, there is no guarantee that seats are still available when a booking is made. This often leads to overbooking or confusion among users.

There is also no clear distinction between roles in traditional approaches. Anyone can claim to organize an event, and there is no approval mechanism to verify or control event visibility.

This project solves these problems by:

Introducing a database-driven system where all events and bookings are stored and managed.

Adding a status field (Pending/Approved/Rejected) to control event visibility.

Implementing seat validation logic in the backend before confirming a booking.

Enforcing role-based access control, ensuring that only authorized users can perform specific actions.

The system ensures that every booking, event creation, and approval step is tracked and processed through defined backend logic, reducing errors and improving reliability.

Existing Approaches

Before this system, event booking was handled without structured validation or role control. The differences become clearer when mapped directly to how this project solves them.

Area	Traditional Method	In This System
Seat Management	Manually tracked, error-prone	Checked in backend (bookings.js) before confirming booking
Ticket Generation	Random or manual IDs	Auto-generated ticket_id with regex validation
Event Visibility	Immediate, no verification	Controlled via status (Pending → Approved)
User Roles	No separation	Enforced using role in Users table
Booking Confirmation	Verbal/manual	Stored in Bookings table with timestamp

In manual systems, there is no guarantee that the number of bookings matches available seats. In this project, before inserting a booking, the backend fetches the event and verifies the seats column. If seats are exhausted, the booking request is rejected.

Similarly, event approval is not controlled in traditional setups. Here, events created from organizer-dashboard.html are stored as Pending and only become visible after admin approval through admin.html.

This shift removes dependency on manual tracking and replaces it with controlled database operations.

Approach / Methodology – Tools and Technologies Used

This project uses a split architecture where each technology is responsible for a clearly defined part of the system.

Technology Usage Mapping

Technology	Where It Is Used	What It Does in This Project
HTML	All pages (login.html, dashboard.html, etc.)	Defines structure of forms, event cards, dashboards
CSS	Styling files	Controls layout of event listings and dashboards
JavaScript (Frontend)	main.js, event scripts	Handles API calls, DOM updates, token storage
Node.js	Backend runtime	Runs server logic
Express.js	Route handling (auth.js, etc.)	Manages API endpoints
MySQL	Database	Stores users, events, bookings
JWT	Auth system	Maintains user session across requests

How Components Work Together (Actual Flow)

User interacts with frontend (click, form submit)

JavaScript sends request using fetch()

Request hits Express route (auth.js, events.js, etc.)

Backend processes logic and queries MySQL

Response is sent back as JSON

Frontend updates UI dynamically

Example: Login Flow

StepAction

- 1 User enters email/password in login.html
- 2 JS sends POST request to /auth/login
- 3 Backend verifies credentials using bcrypt
- 4 JWT token generated
- 5 Token stored in browser (localStorage)
- 6 User redirected to dashboard

Workflow

This section explains how different roles interact with the system step-by-step.

Guest Workflow (No Login)

StepAction

- 1 User opens homepage
- 2 Frontend calls /events
- 3 Backend returns only Approved events
- 4 Events displayed as cards

User Workflow (Booking Process)

StepAction

- 1 User registers via /auth/register
- 2 Logs in → receives JWT
- 3 Opens dashboard → fetches events
- 4 Clicks event → opens details page
- 5 Clicks “Book Ticket”
- 6 Request sent to /bookings with token
- 7 Backend checks seat availability
- 8 Generates ticket_id
- 9 Stores booking in DB
- 10 Confirmation shown to user

Organizer Workflow

StepAction

- 1 Organizer logs in
- 2 Opens organizer-dashboard.html
- 3 Submits event form
- 4 Event stored with status = Pending

Admin Workflow

StepAction

- 1 Admin logs in
- 2 Opens admin.html
- 3 Fetches pending events
- 4 Approves or rejects
- 5 Approved events become visible

Assumptions

While building this system, certain practical assumptions were made to keep the implementation consistent and manageable.

Area	Assumption	How It Reflects in Project
User Data	Users provide valid email during registration	Email is used as unique identifier in auth.js
Roles	Roles (User, Organizer, Admin) are pre-defined	Stored in role column in Users table
Authentication	Token is always sent after login	Frontend stores JWT in localStorage
Event Creation	Organizer enters correct event details	No heavy validation beyond required fields
Seat Count	Seats represent total available tickets	Used directly in booking validation
Booking	One request = one ticket	No bulk booking logic implemented
Admin Control	Admin manually approves events	No automation for approval

These assumptions simplify the system and ensure that each feature works with predictable inputs.

Implementation – Data Handling, Processing, and System Behavior

This section explains how actual data flows through the system, focusing on UI → API → Database interactions.

7.1 Frontend Screens and Behavior

Login Page (login.html)

Element	Purpose
Email Input	Captures user email
Password Input	Captures password
Login Button	Triggers API request

What happens internally:

User clicks login

JavaScript captures input values

Sends POST request to /auth/login

If success → token stored in localStorage

Redirect to dashboard

Register Page (register.html)

Field	Stored In
Name	Users.name
Email	Users.email
Password	Users.password (hashed)

Flow:

Data sent to /auth/register

Password hashed using bcrypt

New record inserted in Users table

Dashboard (dashboard.html)

Feature	Behavior
Event List	Loaded dynamically using /events
Event Card	Displays title, date, location
Click Action	Opens event details page

Only events with status = Approved are shown.

Event Details Page

Component Function

Event Info	Full description
Seats Info	Shows availability
Book Button	Initiates booking

Organizer Dashboard (organizer-dashboard.html)

Field	Stored In
Title	Events.title
Description	Events.description
Date	Events.date
Location	Events.location
Seats	Events.seats

Processing:

POST request to /events

organizer_id taken from JWT

status = Pending

Admin Panel (admin.html)

Feature	Action
Pending Events List	Fetch from backend
Approve Button	Updates status to Approved
Reject Button	Updates status to Rejected

7.2 Backend Route Processing

Auth Routes (auth.js)

Route	Input	Output
/register	name, email, password	success message
/login	email, password	JWT token

Events Routes (events.js)

Route	Function
GET /events	Returns approved events
POST /events	Creates event (Organizer only)

Bookings Routes (bookings.js)

Route	Function
POST /bookings	Creates booking

7.3 Database Interaction

Users Table

Column	Purpose
id	Primary key
name	User name
email	Login credential
password	Hashed password
role	Access control

Events Table

Column	Purpose
id	Event ID
title	Event name
description	Details
date	Schedule
location	Venue
seats	Available seats
organizer_id	Links to Users
status	Approval control

Bookings Table

Column	Purpose
id	Booking ID
user_id	Who booked
event_id	Which event
ticket_id	Unique ticket
booking_time	Timestamp

7.4 Booking Execution Flow (Detailed)

StepInternal Action

- 1 User clicks “Book Ticket”
- 2 Token sent in header
- 3 Backend verifies token
- 4 Event fetched from DB
- 5 Seat count checked
- 6 Ticket ID generated
- 7 Booking inserted
- 8 Seats updated
- 9 Response returned

Solution Design

The project is divided into two main parts: client (frontend) and server (backend). This separation keeps UI handling and data processing independent, which makes debugging and updates easier.

Frontend Structure

File	Purpose
login.html	User login form
register.html	User registration
dashboard.html	Displays approved events
event-details.html	Shows event info and booking option
organizer-dashboard.html	Event creation page
admin.html	Event approval panel

Each page is dedicated to a specific role or action. This avoids complexity and keeps navigation straightforward.

Backend Structure

File	Purpose
server.js	Starts server and connects routes
auth.js	Login and registration logic
events.js	Event fetch and creation
bookings.js	Booking process
admin.js	Event approval/rejection

Routes are separated by functionality so changes in one module don't affect others.

System Flow

User Action → Frontend JS → API Request → Backend Route → Database → Response
→ UI Update

Key Design Choices

Decision	Reason
Separate pages	Easier role-based access
JWT authentication	No need for session storage
Event status control	Prevents unverified events
Modular routes	Clean backend structure

Challenges and Opportunities

Challenges

Issue	How It Was Handled
Seat overbooking	Checked seats before booking
Role control	Verified role using JWT
Data mismatch	Aligned frontend and backend fields
Token handling	Stored in localStorage and sent in headers
Event visibility	Used status field filtering

Opportunities

Area	Improvement
Booking	Add multiple ticket selection
UI	Add search and filters
Admin	Add user management
Validation	Stronger input checks

Reflections on the Project

This project made it clear how frontend actions, backend logic, and database updates are tightly connected. Even a simple action like booking requires validation, database updates, and proper response handling.

Separating logic into modules like auth.js, events.js, and bookings.js made development easier and reduced confusion during debugging.

Recommendations

The current system works reliably for basic event booking, but a few improvements can make it more practical and easier to use.

One improvement is adding a search and filter option on the dashboard. Right now, users can only scroll through events. Adding filters like date or location would make event discovery faster.

Another useful addition is multiple ticket booking. Currently, one booking request creates only one ticket. Allowing users to select quantity would reduce repeated actions and improve usability.

The admin panel can also be extended. At the moment, it only handles event approval. It can be improved by adding:

User management (view users, roles)

Booking overview (total bookings per event)

Input validation can also be strengthened. While basic checks exist, adding stricter validation (like preventing invalid dates or empty fields) would reduce incorrect data entries.

Outcome / Conclusion

The system successfully implements a complete flow for event booking with clear role separation.

Users can:

Register and log in securely

View approved events

Book tickets with proper validation

Organizers can:

Create events through a dedicated dashboard

Submit events for approval

Admins can:

Control which events are visible

Approve or reject event submissions

The most important outcome is that all actions are handled through structured backend logic instead of manual processes. The booking system ensures that seats are checked before confirmation, and all data is stored consistently in the database.

Overall, the project demonstrates how a full-stack application can handle real user interactions, maintain data integrity, and enforce role-based control.

Enhancement Scope

There are several areas where the system can be extended further.

Feature	Description
Payment Integration	Add online payment before booking confirmation
Real-time Updates	Show live seat availability
Notifications	Send booking confirmation messages
QR Tickets	Generate scannable tickets
Mobile Support	Convert into mobile-friendly app

These additions can make the system closer to real-world production platforms.

Link to Code and Executable File

Github- <https://github.com/Ayush365/Event-Management/tree/main>

Research Questions and Responses

How does the system prevent overbooking?

Before inserting a booking, the backend checks the seats value in the Events table. If no seats are available, the request is rejected.

How are users authenticated?

Users log in through /auth/login, and a JWT token is generated. This token is used in all protected requests.

How are events controlled before publishing?

Events are created with status = Pending. Only after admin approval are they shown to users.

How is data consistency maintained?

All operations (event creation, booking, approval) are handled through backend routes that interact directly with the database.

References

[Node.js Documentation](#)

[MySQL Documentation](#)